

Student Grade Predictor

Python Essentials – Evaluated Course Project

Prepared by: Atraiu Roy
VIT Bhopal University
Registration No.: 25BAI11475

A modular command-line Python application for student score prediction and model evaluation.

1. Introduction

The Student Grade Predictor is a command-line Python application demonstrating a complete, modular data-processing and machine-learning workflow. The application uses four input factors—study hours, attendance, sleep hours, and previous score—to estimate a final score. It compares Linear Regression and Random Forest Regression, evaluates them using MAE and R^2 , selects the lower-MAE model, and produces a prediction for user-supplied data.

2. Problem Statement

Students' academic performance can be influenced by measurable study and academic factors. The project demonstrates how Python can be used to collect structured data, train predictive models, evaluate their performance, and provide an estimated final score. The included dataset is synthetic and is intended for educational demonstration rather than real-world academic decision making.

3. Objectives

- Build a reusable Python application rather than a single monolithic script.
- Generate and store a reproducible dataset.
- Train and compare two regression models.
- Validate user input before prediction.
- Evaluate model performance using MAE and R^2 .
- Provide a command-line prediction workflow.
- Include automated tests and documentation.

4. Functional Requirements

- FR1: Generate or load the student dataset.
- FR2: Validate prediction inputs.
- FR3: Train Linear Regression and Random Forest models.
- FR4: Evaluate models using MAE and R^2 .
- FR5: Select the lower-MAE model.
- FR6: Predict a student final score and grade band.
- FR7: Save an actual-vs-predicted visualization.
- FR8: Execute automated validation tests.

5. Non-Functional Requirements

- Usability: terminal prompts and clear error messages.
- Reliability: deterministic dataset generation using a fixed random seed.
- Maintainability: separate modules for data, validation, models, prediction, analytics, and visualization.
- Error handling: invalid numeric ranges are rejected before prediction.
- Resource efficiency: the application uses a compact local CSV dataset and lightweight models.

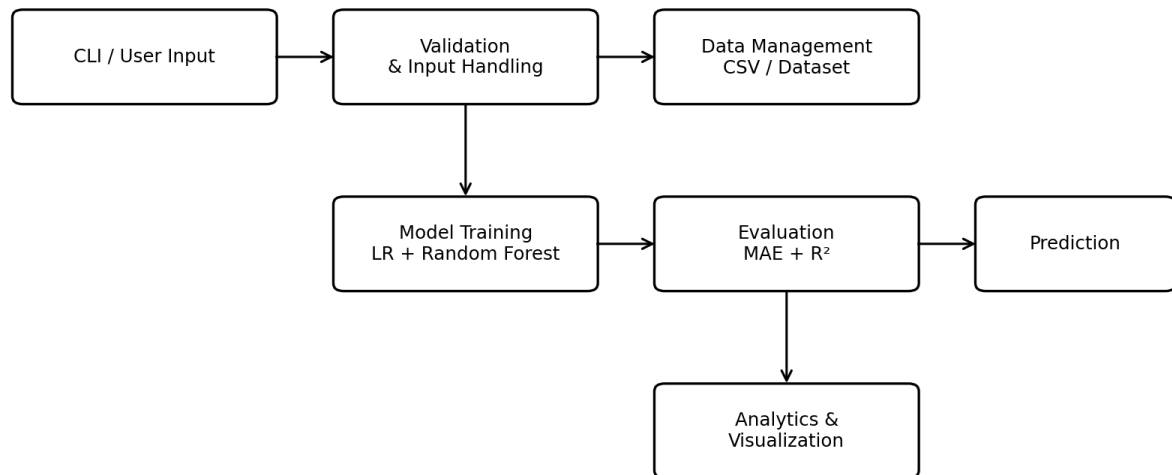
6. System Architecture

The application is organized into a CLI/controller layer and independent functional modules. The data module handles generation/loading, validation handles user input constraints, `model_training` handles model fitting and evaluation, prediction handles score estimation, analytics handles summaries and feature effects, and visualization generates the evaluation chart.

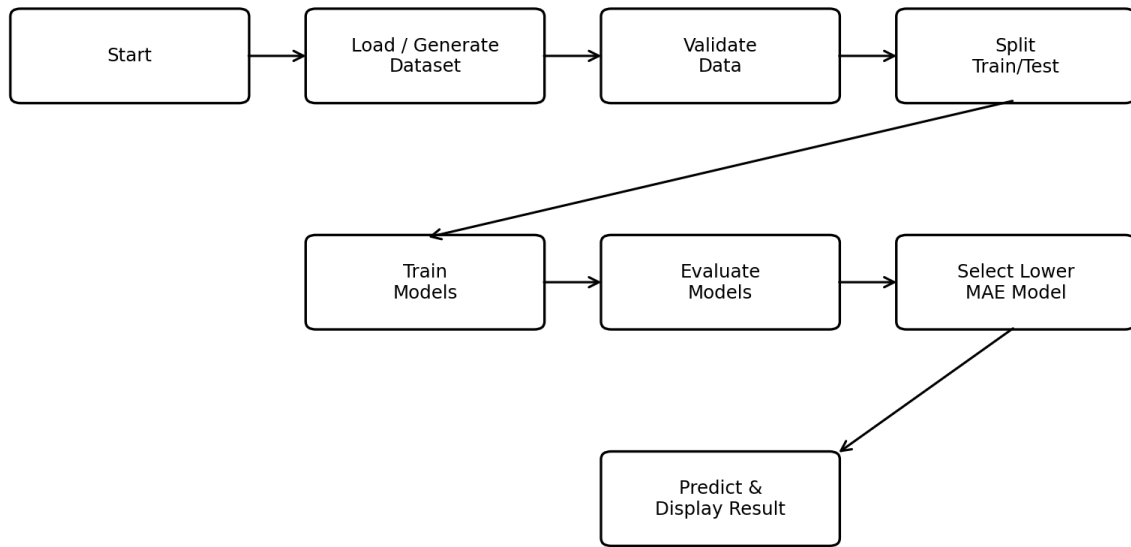
7. Design Diagrams

The following diagrams document the architecture, workflow, use cases, module relationships, and prediction sequence.

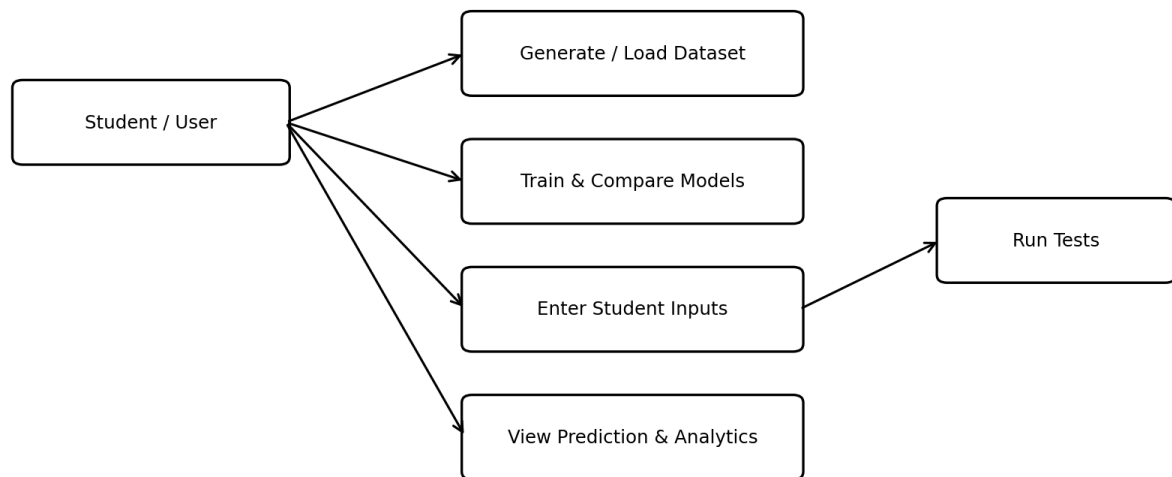
System Architecture



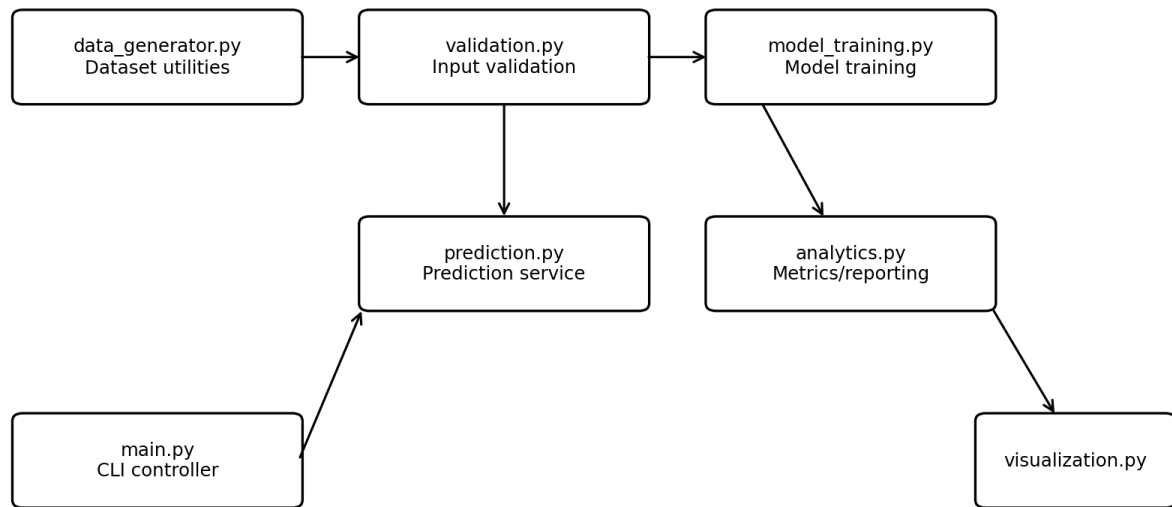
Project Workflow



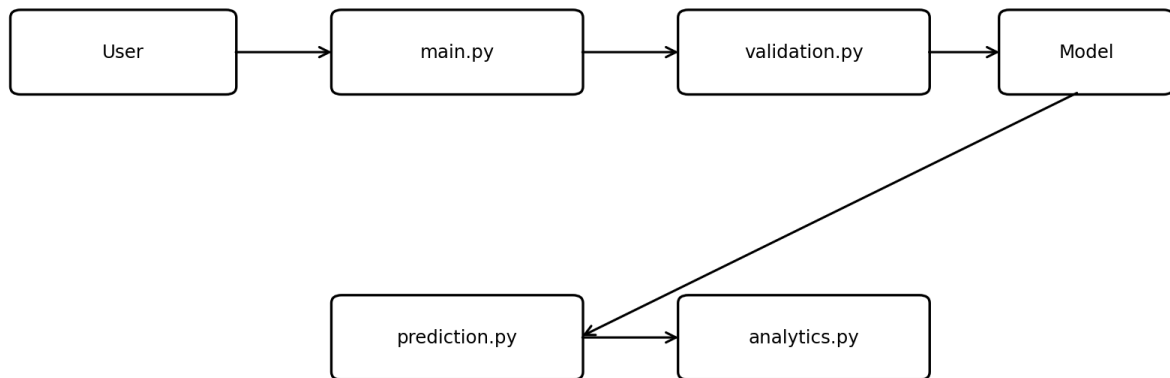
Use Case Diagram



Class / Component Diagram



Prediction Sequence



8. Design Decisions & Rationale

A command-line interface was selected because it is easy to execute in a clean terminal environment. CSV was selected for simple local storage and transparency. Linear Regression provides an interpretable baseline, while Random Forest provides a non-linear comparison model. MAE is used as the primary selection metric because it expresses average prediction error directly in score units.

9. Implementation Details

The dataset contains 300 generated records and four predictor variables. An 80/20 train-test split is used with a fixed random state. Both models are trained on the same split. The final prediction is clipped to the 0–100 score range after input validation. The grade band is derived from score thresholds implemented in prediction.py.

10. Results

In the deterministic local run used while preparing this submission package, Random Forest produced a lower MAE than Linear Regression on the generated test split. The application records both MAE and R^2 and saves an actual-vs-predicted chart. Exact metrics may change if the dataset generation parameters are deliberately changed.

11. Testing Approach

Pytest tests cover grade-band boundaries, prediction range, invalid attendance handling, and dataset generation. The test suite completed successfully with four passing tests in the prepared package.

12. Challenges Faced

The original project was implemented as a single script. The main engineering challenge was separating responsibilities without changing the basic project idea. Another limitation is that the original approach uses synthetic data, so the project documentation explicitly avoids treating the model as a validated real-world predictor.

13. Learnings & Key Takeaways

The project demonstrates modular Python programming, package organization, file I/O, Pandas data handling, NumPy numerical operations, scikit-learn model training, evaluation metrics, visualization, exception handling, testing, and GitHub-oriented documentation.

14. Future Enhancements

Potential future work includes using an ethically sourced real dataset, cross-validation, hyperparameter tuning, additional models, explainability methods, fairness checks, persistent model files, and an optional web interface.

15. References

Python documentation; NumPy documentation; Pandas documentation; Matplotlib documentation; scikit-learn documentation; Pytest documentation. The project repository is maintained on GitHub at <https://github.com/atrainu2007/Student-grade-predictor> .